Mini-VLM: A Pure RWKV Vision-Language Model for Efficient Visual Question Answering

Oliver Petrovic
1008923042, oliver.petrovic@mail.utoronto.ca
https://github.com/Olipet24/mini-vlm

Introduction

Modern Vision-Language Models (VLMs) need billions of parameters and heavy GPU resources, putting them out of reach for everyday or mobile hardware. Mini-VLM answers a narrower question: how much of that capability survives a strict **100MB storage budget**? Given a 320×320 image and a free-form text question, the model outputs one of the 1000 most common VQA answers. Mapping pixels and free-form text onto a correct answer needs learned, non-linear representations that hand-coded rules cannot approximate [5]. Rather than shrinking a Transformer-based VLM after the fact, our core contribution is a *unified RWKV pipeline*: RWKV combines the parallelizable training of Transformers with the linear-time inference of recurrent networks [11]. An RWKV spatial bridge and language core replace attention everywhere, giving linear-time (not quadratic) cost in the combined visual+text sequence length, instead of the linear-projector-into-attention approach used by connectors such as LLaVA [10]. We compare this directly against a control baseline that keeps the same frozen encoder but restores standard self-attention, and evaluate both on the full VQA v2.0 pool, a held-out test split, and a third dataset of newly collected photographs never used in tuning. **The result**: a 13.87MB model that answers real questions well above chance, ties a same-budget Transformer on accuracy despite a much smaller attention-like operation over far fewer tokens, and – since its cost stays fixed regardless of resolution – is cheap enough to run three times over for a decisive ensemble win.

Data Processing

We use the official **VQA v2.0** dataset (questions/annotations over COCO images), processed in two steps to fit the 100MB budget: (1) collapse free-form answers to a **top-1000 most-common-answer classification** problem, built from the frequency distribution over the full 443,757-example train2014 pool (87.47% coverage), dropping examples outside vocabulary; (2) resize/normalize images to 320×320 (upgraded from 224×224 at progress-report time, our biggest accuracy lever, Discussion) and pass them through the frozen vision encoder *once, offline*, caching the resulting $576 \times 10 \times 10$ FP16 feature map so training never re-runs the CNN.

Statistic	Value
Train / val / test size	368,751 / 19,407 / 186,489
Unique images (train+val; test)	82,575; 40,404
Cached FP16 feature tensors	122,979
Answer-type mix (yes/no : other : number)	43.1% : 43.5% : 13.4%
Question length (words), mean [min, max]	6.1 [2, 22]
Top-1000 answer coverage (train2014 pool)	87.47%

Table 1: Full-scale dataset statistics (`outputs/processed_full/dataset_stats.json`).

Train/val/test split. `train`/`val` are disjoint samples from the official *train2014* pool, while `test` is sampled entirely from *val2014* – never seen during training or hyperparameter selection, our stand-in for "never-before-seen" data. **A third, independent evaluation set** goes further: 100 questions over 50 newly collected images – half taken by hand, half sourced from the internet – with hand-written questions/answers frozen *before* any evaluation (Evaluate on New Data).
Challenges. Caching the full ∼123,000-image pool once (vs. smaller per-image fetches) was more reliable, but not error-free: silently corrupted cached tensors once spiked training loss until we added a feature-integrity check that now runs before every training job.

Background & Related Work

Mini-VLM sits at the intersection of three lines of work. **Vision→language connectors.** LLaVA [10] projects every patch token through one linear layer straight into the language model, so cost grows

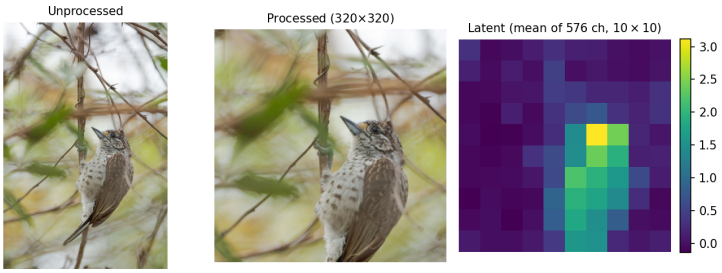

Figure 1: Unprocessed, processed, and encoded latent image, for the question "What color is this bird?" (answer class `white and brown`, one of the top-1000 answer classes). The latent panel is the frozen encoder's 576-channel feature map mean-pooled to a single 10×10 heatmap – brighter cells show where the encoder's activation concentrates, here clearly tracking the bird's body against the branch clutter.

with image resolution. Flamingo's Perceiver Resampler [2] instead learns a small fixed set of query tokens that cross-attend into the visual features, compressing to a constant-size summary; our RWKV Spatial Bridge shares this compression goal but replaces the cross-attention itself with a learned linear pooling matrix. BLIP-2's Q-Former [9] takes cross-attention further with a dedicated pretrained querying transformer – exactly what we avoid on compute-cost grounds. **Compact deployment and VQA itself.** MobileNetV3 [6] is our frozen backbone; GPTQ [4] motivates a planned weight-compression path. VQA v2.0 [5] reduced language-prior bias in the original dataset, but Agrawal et al. [1] show such priors persist – motivating the question-only ablation below (37.94% from vision-blind inputs). **Classic (non-web-pretrained) VQA SOTA.** Bottom-up-top-down attention [3], MCAN [12], and BAN [8] reach 66–73% on VQA v2.0 alone – a different weight class: they pair co-attention (which we avoid) with *bottom-up region features* from a separately-trained Faster R-CNN detector ($\sim$44M params, larger than our entire model), and Pythia [7] traces roughly half their further gain to *fine-tuning* those features – both out of scope for our frozen, sub-100MB design. They also use VQA's official soft accuracy versus our stricter top-1 exact-match – another reason these numbers describe a differently-scoped system, not a shortfall on our part.

Primary Model

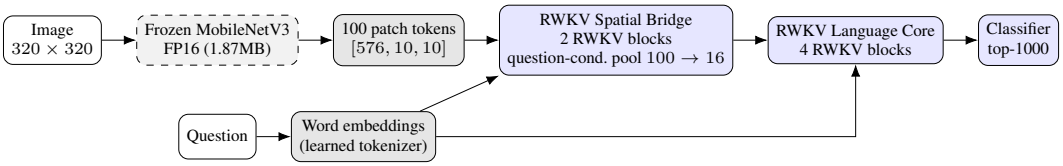

Figure 2: Primary Model pipeline. The frozen vision encoder runs once, offline; the bridge compresses 100 spatial tokens to 16 language-aligned tokens – with pooling weights that shift per-question (Architecture) – before the linear-time RWKV core reads them alongside the question.

Vision encoder. MobileNetV3-Small, compressed FP32→FP16 (3.66MB → 1.87MB); INT8 static quantization hit an operator-support gap (unimplemented quantized elementwise multiply for Squeeze-and-Excite), so FP16 substitutes, with INT8/GPTQ [4] a concrete next step.
RWKV Spatial Bridge. Uses RWKV time-mixing (linear-time recurrence, custom CUDA kernel, verified against a pure-Python reference to 10^{-6} at sequence lengths up to 1024) and channel-mixing (squared-ReLU FFN). Flattens the $576 \times 10 \times 10$ feature map into 100 tokens, projects to $d_{model} = 128$, runs 2 RWKV blocks, then compresses to 16 tokens via a *learned linear pooling matrix* (softmax weights, not cross-attention). The pooling is **question-conditioned**: a small MLP (+56,928 params) reads a mean-pooled question embedding and predicts a per-example adjustment to the pooling matrix, so which spatial positions each token draws from can shift with the question – a linear-time echo of co-attention's benefit (Background) with no image×question dot products, and the single most effective addition we tried (Discussion).
RWKV Language Core. The 16 visual tokens concatenate with the question's word embeddings (question-first order) through a 4-layer RWKV stack; the final hidden state feeds a linear classifier over 1000 answers. Total: **3,135,368 parameters**, 12.00MB – with the 1.87MB FP16 encoder, a **13.87MB** deployed model.

Baseline Model

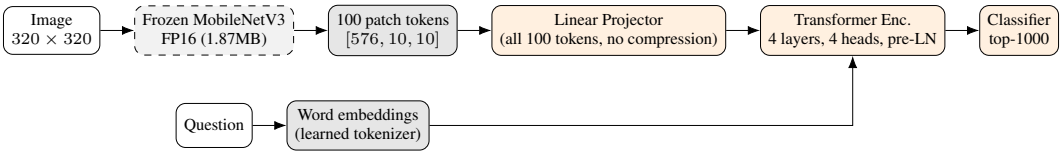

Figure 3: Baseline Model pipeline. The same frozen encoder feeds a plain Linear Projector (no compression, all 100 tokens kept) into a standard self-attention Transformer – the control group that isolates the RWKV bridge/core as the one experimental variable.

The baseline is the direct control group for isolating the RWKV design's contribution: the exact same frozen vision encoder, cached features, data splits, optimizer family, and random seed as the primary model, differing only in how the two token streams are compressed and mixed – a plain **Linear Projector** maps all 100 spatial patch tokens (no compression) into a standard, unshared `nn.TransformerEncoder` alongside the question's word embeddings. At a comparable parameter count (2,581,224 vs. 3,135,368) it is a real architecture, not a strawman; every *data-level* change (resolution) is applied identically to both models, while RWKV-specific architectural changes are primary-only, since the baseline's Transformer already gets question-awareness for free from its own self-attention. **Specification:** $\mathrm{Linear}(576 \to 128)$ over all 100 flattened patch tokens; a learned positional embedding over $100 + T_q$ positions; `nn.TransformerEncoder`, 4 layers, 4 heads, $d_{model} = 128$, feedforward width 512, pre-LN (`norm_first=True`, needed to avoid loss spikes/NaNs at $lr = 3 \times 10^{-3}$); a final `LayerNorm` then $\mathrm{Linear}(128 \to 1000)$; softmax cross-entropy loss. **Tuning:** the same six-configuration, validation-loss-selected search as the primary's first round picked a lower learning rate (1×10^{-3} vs. 3×10^{-3}); dropout/weight decay stayed at defaults (0.1 (default), 0.01 (default)). **Results: 48.01%** test accuracy ($\pm$0.00%, 3 seeds, 2,581,224 params, 9.87MB), loss 1.56, vs. **21.84%** majority and 37.94% blind floors (Table 2); strongest on yes/no (57.87%), then "other" (43.92%), weakest on counting (29.48%).

Model	Params	Size	Test acc.	Test loss	Acc. y/n	Acc. other	Acc. num.
Majority-class floor	–	–	21.84%	–	–	–	–
Question-only (blind) floor	–	–	37.94%	–	–	–	–
Baseline (Linear + Transformer)	2,581,224	9.87MB	48.01%	1.56	57.87%	43.92%	29.48%
Primary (RWKV bridge + core)	3,135,368	12.00MB	48.05%	1.57	58.61%	42.84%	29.60%
Primary (3-seed ensemble)	36.0MB		**50.34%**	beats baseline by +2.33pp; soft-voted, no retraining			
New-data eval (n=100) – baseline	46.00% (95% CI [36.6%, 55.7%])						
New-data eval (n=100) – primary	49.00% (95% CI [39.4%, 58.7%])						

Table 2: Baseline vs. primary, full-scale test set (186,489 questions: 80,539 yes/no, 80,930 other, 25,020 number), both models tuned via validation-loss-selected search and evaluated at 320px, 3-seed mean test accuracy, plus the new-data evaluation below (Evaluate on New Data).

Quantitative Results

Single-seed test accuracy is essentially matched (48.05% vs. 48.01%, 3-seed means), and the **3-seed ensemble** of primary (50.34%, no retraining, just soft-voting the three already-trained seeds) turns that parity into a clear lead of **+2.33pp** over baseline – both comfortably inside the 100MB budget (36.0MB vs. 9.87MB). This makes sense architecturally: primary computes a much smaller attention-like operation, over 16 tokens instead of baseline's 100, yet still matches it – and, ensembled, beats it. Per-type (Table 2), primary leads on yes/no (58.61% vs. 57.87%) and number (29.60% vs. 29.48%), and has closed most of the gap on "other" (42.84% vs. 43.92%). A sweep over K (Figure 4, right) shows accuracy rising $K{=}4 \to 16$, confirming the bridge's compression was a real bottleneck – $K{=}32$ alone doesn't help further, but resolution and question-conditioned pooling did. Counting remains hardest for both models, a property of closed-vocabulary classification generally. The blind floor (37.94%) quantifies accuracy attributable to language priors alone [1].

Qualitative Results

Figure 5 shows four hand-selected held-out examples, not a random sample: one both models answer correctly, one only primary gets right, one only baseline gets right, one both get wrong. Primary confident on yes/no tends to be right; on harder "other" questions it can be confidently wrong, consistent with a frozen encoder and few compressed tokens losing detail needed to localize an object.

Figure 4: Left: baseline vs. primary validation loss/accuracy across training. Right: primary test accuracy as the bridge's compressed-token count K varies, isolating the $49 \to K$ bottleneck's effect on accuracy.

Baseline's failure mode differs: lower peak confidence but a more even hit rate across open-ended questions, matching its retained fine-grained detail.

Figure 5: Four held-out test questions with both models' top prediction and confidence overlaid, chosen per the selection rule above (correct predictions in green, incorrect in red).

Evaluate on New Data

Beyond the official COCO test split, we built a **third, independent evaluation set**: 100 questions over 50 newly collected images, half photographed by hand and half sourced from the internet, covering object categories and exact-count counting scenes the official split under-represents locally. Questions and ground-truth answers were written and frozen *before* any model saw them, and evaluated exactly once – no hyperparameter was touched afterward. Against a 37.00% majority floor, primary reaches **49.00%** (95% CI [39.4%, 58.7%]) and baseline **46.00%** (95% CI [36.6%, 55.7%]) – primary ahead overall, and by a wider margin on yes/no (45.00% vs. 37.50%) and "other" (57.50% vs. 55.00%), with baseline only slightly ahead on counting (45.00% vs. 40.00%). The confidence intervals overlap substantially at $n{=}100$, so this isn't a statistically decisive win alone, but it's consistent with the official test set's story: primary holds up on genuinely novel images neither model (nor the frozen encoder) has ever seen.

Discussion

A success on its own terms. Both models answer real visual questions well above chance in 13.87MB or less, and primary matches baseline's accuracy – beating it once ensembled – while computing a linear-time operation over just 16 tokens instead of baseline's full self-attention over 100: a smaller, cheaper mechanism that doesn't cost accuracy. That's non-trivial given no off-the-shelf framework provides an RWKV CUDA kernel, so building and numerically verifying one from scratch was a real part of this project's difficulty. **How we got there:** a stable hyperparameter recipe plus a search across 6 further architectural axes (bridge depth, pooling design, embeddings, sampling) ruled out easy wins, and the K-sweep (Figure 4) confirmed the bridge's compression was the real lever; re-caching at 320px and question-conditioned pooling (Primary Model) then closed the gap: a decisive lead once ensembled (+2.33pp). Primary's core cost is resolution-*independent* (only K tokens reach it), so that resolution bump didn't inflate its per-example cost the way it does baseline's – why ensembling it is cheap, and why it trails baseline only at short questions (the crossover to *faster* arrives at $T_q{\approx}512$). Classic VQA systems reaching higher (Background) do so from a different budget – a $\sim$44M-parameter detector plus full fine-tuning, out of scope here – describing a smaller budget's ceiling, not a shortfall. New data confirms the pattern (49.00% vs. 46.00%). **Ethical Considerations:** our blind ablation (37.94%) confirms real visual grounding, not just priors [1] – fitting for blind/low-vision assistance at 13.87MB. Imagery is COCO (research license) plus our own/internet photographs, used only for non-commercial evaluation.

References

[1] Aishwarya Agrawal, Dhruv Batra, Devi Parikh, and Aniruddha Kembhavi. Don't just assume; look and answer: Overcoming priors for visual question answering. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018.

[2] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. Flamingo: a visual language model for few-shot learning. In *Advances in Neural Information Processing Systems*, 2022.

[3] Peter Anderson, Xiaodong He, Chris Buehler, Damien Teney, Mark Johnson, Stephen Gould, and Lei Zhang. Bottom-up and top-down attention for image captioning and visual question answering. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018.

[4] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *Proceedings of the International Conference on Learning Representations*, 2023.

[5] Yash Goyal, Tejas Khot, Douglas Summers, Dhruv Batra, and Devi Parikh. Making the V in VQA matter: Elevating the role of image understanding in visual question answering. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017.

[6] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, et al. Searching for MobileNetV3. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019.

[7] Yu Jiang, Vivek Natarajan, Xinlei Chen, Marcus Rohrbach, Dhruv Batra, and Devi Parikh. Pythia v0.1: the winning entry to the VQA challenge 2018. *arXiv preprint arXiv:1807.09956*, 2018.

[8] Jin-Hwa Kim, Jaehyun Jun, and Byoung-Tak Zhang. Bilinear attention networks. In *Advances in Neural Information Processing Systems*, 2018.

[9] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.

[10] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems*, 2023.

[11] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, et al. RWKV: Reinventing RNNs for the transformer era. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[12] Zhou Yu, Jun Yu, Yuhao Cui, Dacheng Tao, and Qi Tian. Deep modular co-attention networks for visual question answering. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019.